

# PYTHON NOTES — LOOPS

## TOPIC: LOOPS

Loops allow a Python program to **repeat code** without us having to write the same instructions again and again.

Loops are useful when we want to:

- Repeat an action
- Count numbers
- Go through items one at a time
- Repeat a task until a condition is met
- Build programs that need repeated user input

Python has two main types of loops:

- `for` loops
- `while` loops

---

## 1. `for` LOOPS

A `for` loop is used to go through items in a sequence or collection one at a time.

### Example

```
for number in [1, 2, 3, 4]:  
    print(number)
```

### Output

```
1  
2  
3  
4
```

The loop takes each item from the collection and stores it in the loop variable one at a time.

---

## 2. `for` LOOPS WITH STRINGS

A `for` loop can also go through the characters of a string.

```
for letter in 'Python':  
    print(letter)
```

### Output

```
P  
y  
t  
h  
o  
n
```

This shows that strings can be looped through one character at a time.

---

## 3. `range()`

`range()` is commonly used with `for` loops to generate a sequence of numbers.

`range(stop)`

```
for number in range(5):  
    print(number)
```

### Output

```
0  
1  
2  
3  
4
```

### Important

The stop value is **not included**.

So:

```
range(5)
```

produces:

```
0, 1, 2, 3, 4
```

---

## 4. range(start, stop)

We can choose where the sequence begins.

```
for number in range(1, 6):  
    print(number)
```

### Output

```
1  
2  
3  
4  
5
```

The stop value `6` is not included.

---

## 5. range(start, stop, step)

We can also control how much the number changes each time.

```
for number in range(2, 11, 2):  
    print(number)
```

### Output

```
2  
4
```

```
6
8
10
```

Here:

- 2 → starting value
- 11 → stopping point, not included
- 2 → step size

---

## 6. COUNTING WITH `for` LOOPS

Loops can be used for counting.

```
for number in range(1, 11):
    print(number)
```

This prints the numbers from 1 to 10.

We can also count backwards:

```
for number in range(10, 0, -1):
    print(number)
```

### Output

```
10
9
8
7
6
5
4
3
2
1
```

## 7. while LOOPS

A `while` loop repeats code **while a condition is true**.

### Example

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

### Output

```
1
2
3
4
5
```

### How it works

1. Python checks the condition.
2. If the condition is `True`, the loop runs.
3. The value changes.
4. Python checks the condition again.
5. The loop stops when the condition becomes `False`.

---

## 8. CHANGING THE VALUE IN A `while` LOOP

A `while` loop often needs something that changes the condition.

Example:

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

The line:

```
number += 1
```

changes the value each time.

Without a proper change, a `while` loop may continue forever.

---

## 9. `while True`

`while True` creates a loop that keeps running until something explicitly stops it.

Example:

```
while True:  
    print('Hello')
```

This keeps running because `True` is always true.

We normally use `break` to stop this type of loop.

---

## 10. `break`

`break` immediately stops the loop.

Example:

```
for number in range(1, 11):  
    if number == 5:  
        break  
  
    print(number)
```

### Output

```
1  
2
```

```
3
4
```

When the number becomes `5`, `break` stops the loop.

## Remember

`break` → stop the entire loop

---

## 11. `continue`

`continue` skips the current iteration and moves to the next iteration.

Example:

```
for number in range(1, 6):
    if number == 3:
        continue

    print(number)
```

## Output

```
1
2
4
5
```

When the number is `3`, Python skips that iteration.

## Remember

`continue` → skip this iteration and keep looping

---

## 12. break VS continue

break

```
break
```

Stops the entire loop.

continue

```
continue
```

Skips only the current iteration.

### Easy memory trick

break = leave the loop

continue = keep the loop going

---

## 13. LOOPS WITH if, elif, AND else

Loops can contain decision-making statements.

Example:

```
for number in range(1, 6):  
    if number == 3:  
        print('Three')  
    else:  
        print(number)
```

This combines:

### Loops + Decision Making

The loop controls repetition, while the if controls the decision.

---

## 14. LOOPING THROUGH LISTS

A `for` loop can go through a list one item at a time.

```
names = ['Chris', 'Joyce', 'Belle']  
  
for name in names:  
    print(name)
```

### Output

```
Chris  
Joyce  
Belle
```

This is one of the most important uses of loops because lists contain multiple values.

---

## 15. LOOPING THROUGH STRINGS AND OTHER COLLECTIONS

We can loop through strings:

```
word = 'Python'  
  
for letter in word:  
    print(letter)
```

We can also use loops with other data structures.

As I continue learning dictionaries, tuples, and sets, I will learn how loops work with those structures too.

---

## 16. MULTIPLICATION TABLE WITH A LOOP

Loops can repeat calculations.

Example:

```
number = 5

for i in range(1, 11):
    print(number, 'x', i, '=', number * i)
```

This creates the multiplication table for `5`.

This combines:

- Variables
- `for`
- `range()`
- Arithmetic
- `print()`

---

## 17. USING `while` FOR REPEATED INPUT

A `while` loop is useful when we do not know exactly how many times a user will repeat an action.

Example:

```
while True:
    answer = input('Type done to stop: ')

    if answer == 'done':
        break
```

The program keeps asking until the user types `done`.

---

## 18. MY `loops.py` PROJECT — NUMBER GUESSING GAME

This is the project I created while learning Loops.

```
while True:
    user = int(input('Guess a number: '))

    if user == 7:
```

```
        print('Correct! You guessed the number!')
        break

    elif user > 7:
        print('Too high! Try again.')

    else:
        print('Too low! Try again.')
```

## How the program works

The program begins with:

```
while True:
```

This allows the guessing process to continue repeatedly.

The user enters a number:

```
user = int(input('Guess a number: '))
```

The `int()` converts the input into an integer.

Then the program checks the guess.

### Correct guess

```
if user == 7:
    print('Correct! You guessed the number!')
    break
```

If the user guesses `7`, the program prints the success message and `break` stops the loop.

### Guess is too high

```
elif user > 7:
    print('Too high! Try again.')
```

The loop continues so the user can guess again.

**Guess is too low**

```
else:  
    print('Too low! Try again.')
```

The loop continues again.

---

## 19. CONCEPTS USED IN MY `loops.py` PROJECT

The project combines:

- `while True`
- `if`
- `elif`
- `else`
- `break`
- `input()`
- `int()`
- Comparison operators
- Repetition

This is a good example of combining **Decision Making and Loops**.

---

## 20. WHY LOOPS ARE IMPORTANT

Without loops, we would have to repeat code manually.

For example:

```
print(1)  
print(2)  
print(3)  
print(4)  
print(5)
```

can be replaced with:

```
for number in range(1, 6):  
    print(number)
```

Loops make programs:

- Shorter
- More efficient
- Easier to modify
- More flexible
- Better for repeated tasks

---

## 21. COMMON LOOP MISTAKES

### Forgetting to change a `while` condition

Example:

```
number = 1

while number <= 5:
    print(number)
```

The condition never changes, so the loop can continue indefinitely.

---

### Forgetting indentation

Correct:

```
for number in range(5):
    print(number)
```

Incorrect:

```
for number in range(5):
print(number)
```

Python uses indentation to show which code belongs inside the loop.

---

## Forgetting that `range()` stops before its end value

```
range(1, 5)
```

produces:

```
1
2
3
4
```

It does not include `5`.

---

## WHAT I CAN DO NOW

After learning Loops, I can:

- ☒ Use `for` loops
  - ☒ Use `while` loops
  - ☒ Use `range()`
  - ☒ Use `while True`
  - ☒ Stop loops with `break`
  - ☒ Skip iterations with `continue`
  - ☒ Count with loops
  - ☒ Loop through strings
  - ☒ Loop through lists
  - ☒ Combine loops with `if`, `elif`, and `else`
  - ☒ Use loops with user input
  - ☒ Repeat tasks until a condition is met
  - ☒ Build guessing games
  - ☒ Build multiplication tables
  - ☒ Use loops as part of larger programs
- 

## MY PROGRESS

Current Level: Loops ☒

## Topics Learned

- ✓ `for` loops
- ✓ `while` loops
- ✓ `range()`
- ✓ `break`
- ✓ `continue`
- ✓ `while True`
- ✓ Counting with loops
- ✓ Looping through collections
- ✓ Combining loops with conditions
- ✓ Using loops with user input

## Project

- ✓ `loops.py` — Number Guessing Game
- 



# LOOPS COMPLETE

Loops taught me how to make Python repeat instructions efficiently.

I can now choose between `for` and `while` loops depending on the problem, stop loops with `break`, skip iterations with `continue`, generate number sequences with `range()`, and combine loops with conditions and user input.

 **LOOPS** →  **COMPLETE**

**Next major topic: LISTS AND LIST OPERATIONS** 